



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ÉTABLISSEMENT ALKENDI

RAPPORT DE STAGE DE FIN D'ÉTUDES

Pour l'obtention du Brevet de Technicien Supérieur (BTS)

Filière : Développement d'Applications Informatiques (DAI)

Conception, Architecture et Développement d'un Écosystème Numérique Unifié : Service Location, Service ERP et Service CRM

Réalisé par :
Rayan El Habib

Entreprise d'accueil :
Prestige Maroc

Année Universitaire : 2025 / 2026

Dédicace

Je dédie ce travail à mes parents, pour leur soutien inconditionnel et leurs sacrifices tout au long de mes études. À mes amis et à tous ceux qui ont cru en moi et m'ont encouragé à aller toujours plus loin dans le domaine passionnant de l'ingénierie logicielle.

Remerciements

Je tiens à exprimer ma profonde gratitude à Monsieur , mon encadrant pédagogique à l'établissement Alkendi, pour ses précieux conseils, sa disponibilité constante et la rigueur scientifique qu'il m'a inculquée tout au long de ma formation en BTS DAI. Mes vifs remerciements s'adressent également à l'équipe dirigeante de **Prestige Maroc**, et plus particulièrement à mon encadrant industriel, pour m'avoir fait confiance et m'avoir intégré au sein d'un environnement professionnel stimulant. Enfin, je remercie chaleureusement les membres du jury pour l'honneur qu'ils me font en évaluant ce rapport de fin d'études.

Résumé

Ce projet de fin d'études porte sur la conception et le développement d'un Écosystème Numérique Unifié (ERP, CRM, Location B2C) pour l'agence de location de voitures Prestige Maroc. L'objectif majeur était de résoudre les problématiques de gestion logistique (conflits de réservation, facturation manuelle) en migrant vers une architecture moderne Cloud-Native. La solution adoptée est une architecture **Micro-services API-First** basée sur la Stack **MERN** (MongoDB, Express, React, Node.js). Le projet intègre des concepts d'ingénierie avancés : Sécurité (RBAC, JWT), optimisation (Redis, Verrouillage Optimiste), et DevOps (Docker, CI/CD). Ce système permet à l'entreprise d'optimiser sa gestion quotidienne de manière totalement numérisée.

Mots-clés : BTS DAI, Microservices, MERN, Cloud, Docker, ERP, CRM.

Abstract

This end-of-studies project focuses on the design and development of a Unified Digital Ecosystem (ERP, CRM, B2C Rental) for the car rental agency Prestige Maroc. The primary objective was to solve logistical management issues (booking conflicts, manual billing) by migrating to a modern Cloud-Native architecture. The adopted solution is an **API-First Microservices** architecture based on the **MERN** Stack (MongoDB, Express, React, Node.js). The project integrates advanced engineering concepts : Security (RBAC, JWT), Database Optimization (Redis, Optimistic Locking), and DevOps (Docker, CI/CD). This system enables the company to optimize its daily management in a fully digitalized way.

Keywords : BTS DAI, Microservices, MERN, Cloud, Docker, ERP, CRM.

Liste des abréviations

- **API** : Application Programming Interface
- **BTS DAI** : Brevet de Technicien Supérieur - Développement d'Applications Informatiques
- **CRM** : Customer Relationship Management
- **ERP** : Enterprise Resource Planning
- **JWT** : JSON Web Token
- **MCD / MLD** : Modèle Conceptuel de Données / Modèle Logique de Données
- **MERN** : MongoDB, Express, React, Node.js
- **RBAC** : Role-Based Access Control
- **UI / UX** : User Interface / User Experience

Glossaire

- **Cloud-Native** : Application conçue spécifiquement pour fonctionner dans un environnement Cloud (ex : AWS, Google Cloud).
- **NoSQL** : Modèle de base de données non relationnel, offrant une grande flexibilité et évolutivité.
- **Race Condition** : Anomalie logicielle survenant lorsque plusieurs processus tentent de modifier la même donnée simultanément.
- **Webhook** : Mécanisme permettant à une application tierce (comme Stripe) d'envoyer des informations en temps réel à notre serveur.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des abréviations	v
Glossaire	vi
Introduction générale	1
1 Présentation de l'entreprise d'accueil	3
1.1 Présentation détaillée de Prestige Maroc	3
1.2 Environnement Socio-économique et Concurrence	3
1.3 Organigramme et Structuration Interne	4
1.4 Place du projet dans l'entreprise	4
2 Étude de l'existant et analyse du besoin	5
2.1 Critique approfondie de l'existant	5
2.2 Les problèmes et conséquences de l'existant	5
2.3 Analyse Stratégique : Matrice SWOT	6
2.4 Analyse des besoins	6
2.4.1 Besoins fonctionnels	6
2.4.2 Besoins non fonctionnels	7
3 Cahier des charges	8
3.1 Objectif général du projet	8
3.2 Acteurs du système	8
3.3 Règles de gestion métier	8
3.4 Contraintes Légales et Normes de Sécurité	9

3.4.1	Conformité RGPD (Règlement Général sur la Protection des Données)	9
3.4.2	Sécurité Bancaire (Norme PCI-DSS)	9
4	Analyse fonctionnelle et conception	10
4.1	Diagramme de Cas d'Utilisation (UML)	10
4.2	Diagramme de Séquence : Processus de Réservation	10
4.3	Modèle Conceptuel de Données (MCD - Merise)	11
4.4	Modèle Logique de Données (MLD)	11
4.5	Diagramme d'Activité : Gestion des Annonces	11
4.6	Diagramme de Classes (UML)	12
4.7	Architecture Générale du Système (Microservices)	12
5	Choix techniques et outils utilisés	14
5.1	Front-End : React.js, Redux et PWA	14
5.2	Back-End : Node.js, Express et Architecture Microservices	15
5.3	Base de données : MongoDB (NoSQL vs SQL)	15
5.4	Outils de développement et Méthodologie	16
5.5	Méthodologie de travail : Agile Scrum	16
5.6	Planification et Diagramme de Gantt	16
6	Réalisation du projet	17
6.1	Architecture des Microservices et Dépôts (Repositories)	17
6.1.1	1. Le Frontend : enterprise-portal	17
6.1.2	2. Le Backend B2C : service-location	17
6.1.3	3. Le Backend Clients : service-crm (Customer Relationship Management)	18
6.1.4	4. Le Backend Gestion : service-erp (Enterprise Resource Planning)	18
6.2	Explication des Principales Pages et Fonctionnalités	18
6.2.1	Page d'Accueil et Liste des Biens (Véhicules)	19
6.2.2	Page de Détail d'un Bien	19
6.2.3	Page de Connexion / Inscription (Sécurité JWT)	20
6.2.4	Formulaire de Réservation et Paiement (Webhooks)	20
6.2.5	Tableau de bord Administrateur (Dashboard)	20
6.2.6	Gestion des Annonces et Facturation (ERP)	20
6.2.7	Page Contact et Support Client	21
6.3	DevOps : Intégration et Déploiement Continus (CI/CD)	21

7 Tests et validation	22
7.1 La Pyramide des Tests	22
7.2 Tests de Charge et de Performance	23
7.3 Tests de Sécurité (DevSecOps)	23
7.4 Cas de Tests (Tableau Récapitulatif)	24
Conclusion générale	25
Bibliographie et webographie	27
Annexes	28

Table des figures

4.1	Diagramme de Cas d'Utilisation	10
4.2	Architecture Microservices de la plateforme	12
6.1	Page d'accueil du portail B2C (Prestige Maroc)	19

Liste des tableaux

2.1	Matrice SWOT de Prestige Maroc	6
7.1	Cas de tests de l'application	24
7.2	Extrait du dictionnaire de données	28
7.3	Extrait des routes de l'API RESTful	28

Introduction générale

Le secteur de la location de véhicules au Maroc, particulièrement dynamisé par la relance du tourisme international et l'expansion des zones d'affaires, connaît une mutation sans précédent. Face à l'émergence des plateformes mondiales de réservation (Booking, Expedia), les agences locales se doivent de digitaliser leurs processus pour survivre et rester compétitives. Cependant, une proportion alarmante d'agences continue de s'appuyer sur des méthodologies traditionnelles. C'est le cas de **Prestige Maroc**, une agence de location de renom, qui se trouvait freinée dans son expansion par une gestion logistique et administrative obsolète, reposant quasi-exclusivement sur des carnets de réservation manuscrits et des fichiers tableurs (Excel) isolés.

Le présent projet de fin d'études s'inscrit dans le cadre de l'obtention du diplôme de **Brevet de Technicien Supérieur (BTS) en Développement d'Applications Informatiques (DAI)** au sein de l'Établissement Alkendi. L'objectif fondamental qui m'a été confié était de concevoir, architecturer et déployer un **Écosystème Numérique Unifié**. Plus qu'un simple site web "vitrine", ce projet vise à remplacer la totalité du système d'information de l'entreprise par une plateforme web transactionnelle B2C complète, couplée à un outil logiciel de gestion d'entreprise (ERP/CRM) pour l'administration.

Afin de garantir la pérennité, la robustesse et l'évolutivité de ce système face à une potentielle charge de milliers d'utilisateurs simultanés en haute saison, nous avons écarté les architectures monolithiques traditionnelles. J'ai pris la décision stratégique d'implémenter une **Architecture Microservices** moderne, propulsée par la pile technologique **MERN** (MongoDB, Express, React, Node.js).

Pour mener à bien cette ingénierie complexe dans les délais impartis, le projet a été piloté selon la méthodologie **Agile Scrum**, garantissant une livraison continue de fonctionnalités testées.

Ce rapport détaillera avec précision l'ensemble du cycle de vie du développement logiciel (SDLC). Nous débuterons par la présentation du contexte d'accueil et de l'entreprise (Chapitre 1), suivie de l'étude critique de l'existant (Chapitre 2). Nous dresserons ensuite un cahier des charges rigoureux (Chapitre 3) avant d'aborder la modélisation UML et l'analyse fonctionnelle (Chapitre 4). Les justifications de nos choix technologiques (Chapitre 5) introduiront la partie cœur de ce rapport : la réalisation concrète

des microservices (Chapitre 6). Enfin, nous validerons l'intégrité de notre système par une campagne de tests exhaustifs et l'implémentation de pipelines d'intégration continue (Chapitre 7).

Chapitre 1

Présentation de l'entreprise d'accueil

1.1 Présentation détaillée de Prestige Maroc

Nom de l'entreprise : Prestige Maroc

Secteur d'activité : Location de véhicules (Tourisme, Utilitaires, Longue durée)

Prestige Maroc est une agence locale spécialisée dans la location de véhicules pour les particuliers (B2C) et les entreprises (B2B). Fondée il y a près d'une décennie, l'entreprise s'est progressivement forgée une excellente réputation sur le marché national grâce à une stratégie axée sur le renouvellement constant de sa flotte et un service client de proximité.

Ses missions principales s'articulent autour de trois axes stratégiques :

- **La Mobilité Touristique :** Mise à disposition de véhicules de tourisme (citadines, SUV) pour une clientèle internationale et locale, particulièrement lors des hautes saisons estivales.
- **Le Leasing B2B :** Contrats de location longue durée (LLD) destinés aux entreprises locales souhaitant externaliser la gestion de leur parc automobile.
- **Le Service Premium :** Offre de transferts aéroportuaires et de location avec chauffeur privé pour une clientèle VIP.

1.2 Environnement Socio-économique et Concurrence

Le marché de la location de voitures au Maroc est hautement fragmenté. Prestige Maroc évolue dans un environnement concurrentiel dense, face à :

1. **Les multinationales franchisées** (Hertz, Avis, Sixt) qui captent la majorité des flux aéroportuaires grâce à leurs systèmes de réservation mondiaux.
2. **Les agences locales indépendantes** qui se battent principalement sur une guerre des prix, souvent au détriment de la qualité du service.

Pour se démarquer de ces deux extrêmes, Prestige Maroc mise sur un positionnement intermédiaire : offrir la qualité de service d'une multinationale au prix d'une agence locale. Cependant, cette ambition est freinée par un manque cruel d'outils digitaux.

1.3 Organigramme et Structuration Interne

L'entreprise est structurée de manière agile avec plusieurs pôles interdépendants :

- **Direction Générale** : Pilotage de la stratégie globale, gestion des partenariats B2B et validation des grilles tarifaires saisonnières.
- **Service Commercial** : Point de contact direct avec le client. Gère l'accueil physique, la négociation, la rédaction des contrats de location et la fidélisation.
- **Service Logistique (Parc automobile)** : Responsable du cycle de vie physique des véhicules. S'occupe de l'entretien mécanique régulier, du nettoyage, et surtout de la vérification méticuleuse de l'état des véhicules lors des phases de Check-in (retour) et Check-out (départ).
- **Service Comptabilité et Finance** : Assure le recouvrement des factures, le suivi des cautions bancaires, le paiement des fournisseurs (garages, assurances) et les déclarations fiscales mensuelles.

1.4 Place du projet dans l'entreprise

Mon stage a été réalisé en tant que Développeur Full-Stack et Chef de Projet technique, directement rattaché à la Direction Générale. Mon rôle ne consistait pas simplement à "créer un site web", mais à mener à bien une véritable **transformation digitale systémique**. L'objectif était d'auditer les processus manuels des différents services (Commercial, Logistique, Comptabilité) pour concevoir une solution logicielle sur-mesure (ERP/CRM) capable de centraliser, sécuriser et automatiser 100% des flux de travail de l'entreprise.

Chapitre 2

Étude de l'existant et analyse du besoin

2.1 Critique approfondie de l'existant

Avant le lancement de ce projet, la gestion de l'agence Prestige Maroc s'appuyait quasi-exclusivement sur des processus manuels et des outils bureautiques obsolètes. Le workflow typique se déroulait de la manière suivante :

1. Le client contacte l'agence par téléphone, WhatsApp, ou s'y déplace physiquement.
2. L'agent commercial doit interrompre ses tâches pour consulter un fichier "Excel Partagé" (souvent corrompu ou verrouillé par un autre utilisateur) ou vérifier un tableau blanc physique dans l'arrière-boutique afin de s'assurer de la disponibilité d'un véhicule.
3. Si le véhicule est disponible, le commercial rédige le contrat sur papier carbone.
4. La facture est saisie manuellement sur Microsoft Word en recopiant les informations, puis imprimée.
5. Le suivi du kilométrage et des vidanges est noté dans un cahier physique tenu par le chef de parc.

2.2 Les problèmes et conséquences de l'existant

Ce fonctionnement archaïque présente des limites majeures qui freinent gravement la croissance de l'entreprise :

- **Conflits de réservation (Surbooking) :** C'est le problème le plus critique. Si deux commerciaux consultent le tableau Excel au même moment et promettent le même véhicule à deux clients différents pour les mêmes dates, cela crée une insatisfaction client catastrophique.

- **Lenteur administrative et perte de productivité** : La double saisie des informations (sur le contrat papier, puis sur la facture Word, puis dans le registre comptable) prend en moyenne 25 minutes par client.
- **Risques liés à la sécurité des données** : L'absence d'une base de données centralisée (Cloud) expose l'entreprise à une perte totale de son historique client en cas de vol d'ordinateur, de crash de disque dur ou d'incendie. De plus, les données personnelles des clients ne sont pas protégées conformément aux standards modernes (RGPD).
- **Manque de visibilité financière (Reporting)** : La direction est incapable de connaître instantanément le chiffre d'affaires du mois en cours ou le taux de rotation exact de la flotte sans passer des heures à consolider les différents fichiers Excel.
- **Absence de canal de vente digital** : En 2026, l'absence d'un site web transactionnel signifie que Prestige Maroc perd quotidiennement des clients (touristes internationaux) qui réservent exclusivement en ligne et paient par carte de crédit avant même d'arriver dans le pays.

2.3 Analyse Stratégique : Matrice SWOT

Pour justifier la pertinence de la digitalisation, j'ai réalisé une analyse **SWOT** :

Forces (Strengths)	Faiblesses (Weaknesses)
- Flotte récente. - Bonne réputation.	- Gestion sur papier et Excel. - Risques d'erreurs humaines.
Opportunités (Opportunities)	Menaces (Threats)
- Digitalisation du secteur. - Paiement en ligne.	- Concurrence des plateformes (Booking). - Perte de données sans Cloud.

TABLE 2.1 – Matrice SWOT de Prestige Maroc

2.4 Analyse des besoins

2.4.1 Besoins fonctionnels

Le système informatique doit impérativement permettre :

- **Côté Client** : L'inscription/connexion, l'affichage du catalogue de voitures, la recherche avec filtres (prix, marque), et la réservation avec paiement sécurisé.

- **Côté Admin (ERP/CRM) :** La gestion des annonces (ajouter/supprimer), la gestion des utilisateurs, la génération automatique de devis/factures PDF, et un tableau de bord analytique.

2.4.2 Besoins non fonctionnels

- **Sécurité :** Protection des données personnelles (RGPD) et cryptage des mots de passe.
- **Ergonomie :** Interface simple, moderne et parfaitement "Responsive Design" (adaptée aux smartphones).
- **Fiabilité :** Le site doit être capable de gérer un grand nombre de connexions simultanées sans planter.

Chapitre 3

Cahier des charges

3.1 Objectif général du projet

Concevoir une plateforme de location web B2C, couplée à un back-office (ERP et CRM) pour automatiser 100% des flux logistiques et financiers de Prestige Maroc.

3.2 Acteurs du système

- **Le Visiteur** : Peut uniquement consulter le catalogue des véhicules.
- **Le Client** : Utilisateur inscrit, peut réserver, payer et consulter son historique.
- **Le Commercial** : Gère les contrats, valide les réservations et met à jour le statut des véhicules.
- **L'Administrateur (Directeur)** : A tous les droits, y compris la modification des taux de TVA et l'accès au bilan financier.

3.3 Règles de gestion métier

La modélisation du système repose sur des règles de gestion métier strictes, définies en collaboration avec la direction financière de l'agence :

- **RG-01 (Authentification)** : Un visiteur doit obligatoirement créer un compte et être authentifié via un Token JWT pour accéder à l'interface de paiement.
- **RG-02 (Conflit et Disponibilité)** : Un véhicule ne peut être verrouillé pour un client que si son statut est explicitement "Disponible". En cas de requêtes simultanées à la même milliseconde pour le même véhicule, le système doit rejeter la seconde requête via un mécanisme de contrôle de concurrence (Verrouillage Optimiste).
- **RG-03 (Paiement et Acompte)** : Pour confirmer une réservation, le client doit s'acquitter en ligne d'un acompte équivalent à 30% du montant total. Le

solde restant sera facturé sur place.

- **RG-04 (Intégrité Fiscale)** : Une facture générée au format PDF est immuable. Elle est horodatée et se voit attribuer un numéro de série séquentiel. Aucune modification ou suppression de facture n'est permise au niveau de l'interface, conformément à la loi des finances.

3.4 Contraintes Légales et Normes de Sécurité

3.4.1 Conformité RGPD (Règlement Général sur la Protection des Données)

Étant donné que la plateforme, accessible aux touristes européens, récolte des données personnelles sensibles (Nom, Prénom, Email, Historique de géolocalisation), elle doit impérativement respecter le RGPD :

- **Consentement Actif** : Présence obligatoire d'une case à cocher non pré-remplie "J'accepte les CGU et la politique de confidentialité" lors du processus d'inscription.
- **Droit à l'oubli** : L'utilisateur a la possibilité de supprimer de manière autonome et définitive son compte depuis son espace profil.
- **Anonymisation des Données** : Si un client demande la suppression de son compte, ses factures passées sont conservées par l'ERP pour des raisons comptables, mais ses informations nominatives (Nom, Prénom) sont écrasées et remplacées par un *hash* irréversible.

3.4.2 Sécurité Bancaire (Norme PCI-DSS)

La manipulation des données de cartes de crédit est soumise à la très stricte norme internationale **PCI-DSS**. Afin d'exonérer Prestige Maroc de cette lourde responsabilité juridique, nous avons pris la décision de ne **jamais** stocker ni faire transiter les numéros de carte de crédit (PAN) sur notre serveur **service-location** ou dans notre base MongoDB. Tout le processus d'encaissement est déporté de manière sécurisée vers l'API de **Stripe**, qui garantit un chiffrement de bout en bout (AES-256) et une conformité 3D-Secure 2.0.

Chapitre 4

Analyse fonctionnelle et conception

4.1 Diagramme de Cas d'Utilisation (UML)

Afin de modéliser les interactions entre les acteurs et le système, nous avons élaboré le diagramme de cas d'utilisation suivant :

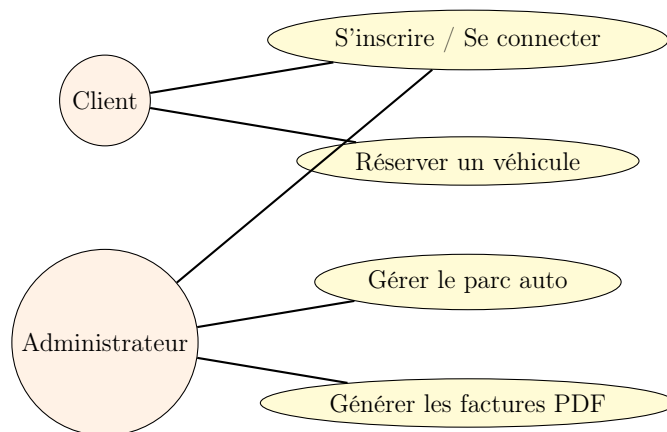


FIGURE 4.1 – Diagramme de Cas d'Utilisation

Explication : Ce diagramme illustre clairement la séparation des privilèges (RBAC). Le client interagit avec la partie Front-Office (réservation), tandis que l'administrateur gère le Back-Office (véhicules, facturation).

4.2 Diagramme de Séquence : Processus de Réservation

Ce diagramme détaille l'échange de messages temporels lors du paiement :

- Le **Client** envoie une requête de réservation au **Front-End React**.
- Le **Front-End** interroge le **Backend Node.js** pour vérifier la disponibilité.

- Le **Backend** consulte la base **MongoDB**.
- Si disponible, Node.js contacte l'API **Stripe** pour le paiement.
- Retour de la confirmation au client et génération de la facture PDF.

Explication : La modélisation de cette séquence est cruciale pour identifier le moment exact où le "Verrouillage Optimiste" doit être déclenché pour éviter le surbooking.

4.3 Modèle Conceptuel de Données (MCD - Merise)

Bien que MongoDB soit orienté documents (NoSQL), la structuration conceptuelle Merise permet d'identifier les entités fondamentales :

- **Entité CLIENT** (ID_Client, Nom, Prenom, Email, Password)
- **Entité VEHICULE** (Immatriculation, Marque, Modèle, Prix_Jour, Statut)
- **Relation RESERVER** (Date_Debut, Date_Fin, Montant_Total)

Règle de cardinalité : Un Client peut effectuer (0,n) réservations, mais une réservation concerne exactement (1,1) client et (1,1) véhicule.

4.4 Modèle Logique de Données (MLD)

Traduction du MCD pour notre architecture NoSQL (Collections JSON) :

- **Users_Collection** : { _id, name, email, password, role }
- **Cars_Collection** : { _id, brand, model, pricePerDay, isAvailable }
- **Bookings_Collection** : { _id, user_id (Ref Users), car_id (Ref Cars), startDate, endDate, totalAmount }

4.5 Diagramme d'Activité : Gestion des Annonces

Le flux de travail (Workflow) d'un Administrateur gérant la flotte automobile :

- **Étape 1** : L'administrateur se connecte (Vérification JWT).
- **Étape 2** : Il saisit les informations d'un nouveau véhicule.
- **Étape 3** : L'image du véhicule est uploadée vers le serveur de stockage (Amazon S3).
- **Étape 4** : Si l'upload réussit, les métadonnées sont sauvegardées dans MongoDB.
- **Étape 5** : Le catalogue React se met à jour instantanément.

Explication : Le diagramme d'activité met en lumière les conditions logiques ("Si upload réussit"). Il permet d'anticiper la gestion des erreurs (ex : afficher un message si le serveur S3 est hors ligne).

4.6 Diagramme de Classes (UML)

Bien que MongoDB soit orienté document, la modélisation objet (POO) via le Diagramme de Classes reste pertinente pour structurer le code backend (Modèles Mongoose) :

- **Classe Utilisateur** : Propriétés (id, nom, prenom, role). Méthodes : `login()`, `register()`, `updateProfile()`.
- **Classe Vehicule** : Propriétés (immatriculation, marque, prix, statut). Méthodes : `checkDisponibilite()`, `bloquerVehicule()`.
- **Classe Reservation** : Propriétés (dateDebut, dateFin, montant). Méthodes : `calculerTotal()`, `validerPaieement()`.

Explication : Ce diagramme définit la structure des objets manipulés par l'application. L'héritage n'a pas été utilisé ici, privilégiant la composition pour plus de flexibilité dans l'API.

4.7 Architecture Générale du Système (Microservices)

Contrairement à une architecture classique (Monolithe), le système est divisé en 4 microservices indépendants, permettant à Prestige Maroc de mettre à jour la comptabilité sans risquer de casser le système de réservation :

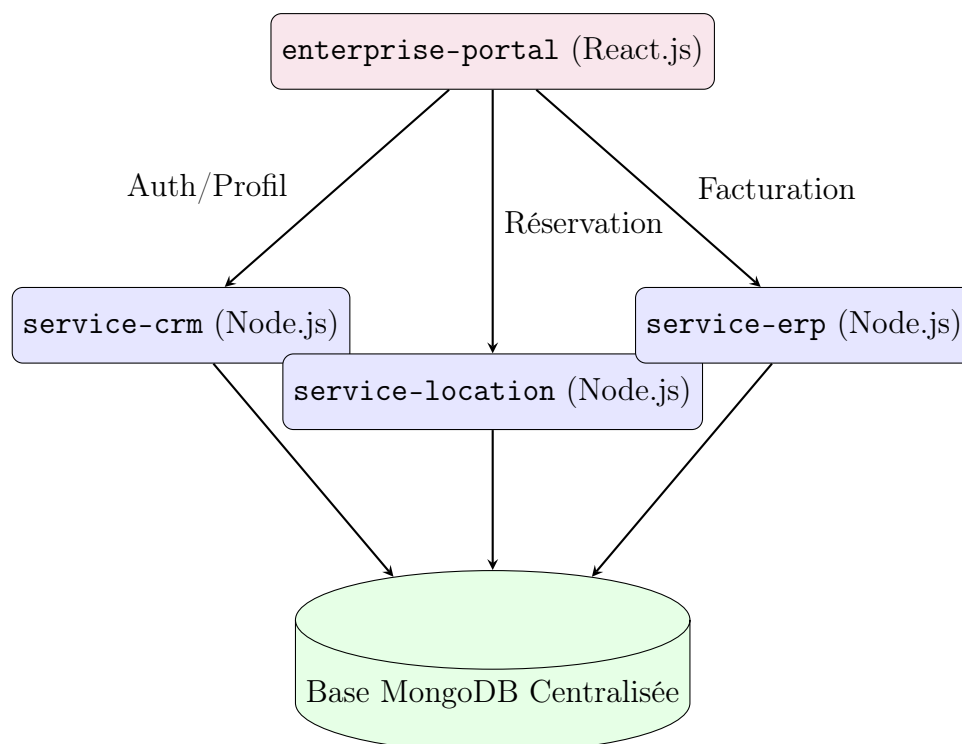


FIGURE 4.2 – Architecture Microservices de la plateforme

Explication :

- Le **Portail Client/Admin** (*enterprise-portal*) est l'unique point d'entrée visuel.
- Lorsqu'un client se connecte, la requête est routée vers le **service-crm** qui vérifie l'identité.
- Lorsqu'il loue une voiture, la requête part vers le **service-location** (qui utilise Redis pour afficher la flotte ultra-rapidement).
- Lorsqu'une facture doit être générée, le portail contacte le **service-erp**, qui exécute la tâche lourde en arrière-plan de manière asynchrone.

Chapitre 5

Choix techniques et outils utilisés

Pour répondre aux exigences de performance et de scalabilité imposées par le cahier des charges, j'ai opté pour la **Stack MERN** (MongoDB, Express, React, Node.js). Le choix de ces technologies n'est pas fortuit, il résulte d'une analyse comparative rigoureuse avec d'autres solutions du marché.

5.1 Front-End : React.js, Redux et PWA

React a été choisi pour sa capacité à créer des "Single Page Applications" (SPA) ultra-rapides. Le DOM Virtuel de React permet de recharger uniquement les composants modifiés (comme le changement de statut d'un véhicule) sans rafraîchir toute la page web, offrant une fluidité proche d'une application de bureau.

- **Approche Mobile-First et PWA** : En 2026, plus de 70% des réservations B2C se font sur smartphone. L'interface a été conçue en "Mobile-First". De plus, elle a été transformée en **PWA (Progressive Web App)**. Les clients peuvent "installer" le site sur leur écran d'accueil iOS ou Android sans passer par l'App Store, tout en bénéficiant d'un mode hors-ligne basique grâce aux Service Workers.
- **Pourquoi pas Angular ou Vue.js ?** Angular est un framework extrêmement lourd, adapté aux gigantesques systèmes bancaires, mais il impose une architecture rigide (TypeScript, RxJS) qui ralentit le prototypage. Vue.js est plus léger, mais l'écosystème communautaire de React reste inégalé.
- **Gestion d'état global (State Management)** : Grâce à **Redux Toolkit**, j'ai pu centraliser les données de l'utilisateur (Token JWT, profil, contenu du panier de réservation). Cela permet à n'importe quel composant de l'application d'accéder instantanément à ces informations sans avoir à les faire passer manuellement de parent à enfant (*Prop Drilling*).

5.2 Back-End : Node.js, Express et Architecture Microservices

Le backend est le moteur logique de l'application. Plutôt que d'opter pour une approche "Monolithique" (un seul gros serveur), j'ai opté pour une **Architecture Microservices**.

- **Pourquoi Node.js plutôt que PHP (Laravel) ou Java (Spring Boot) ?**

Le cœur du métier de la location implique de nombreuses opérations asynchrones (requêtes vers la base de données, appels à l'API bancaire Stripe, envoi d'emails). Node.js, grâce à son architecture événementielle non-bloquante (Event Loop), est capable de traiter des milliers de requêtes simultanées sur un seul *thread*, là où PHP aurait dû ouvrir un *thread* par requête, saturant rapidement la mémoire du serveur.

- **Avantage des Microservices :** En séparant l'application en **service-location**, **service-crm** et **service-erp**, si le service de génération de factures (ERP) plante à cause d'une surcharge mémoire, le système de réservation en ligne continue de fonctionner parfaitement. De plus, cela permet d'utiliser le même langage (JavaScript/TypeScript) sur toute la pile (Full-Stack), réduisant considérablement la friction cognitive lors du développement.

5.3 Base de données : MongoDB (NoSQL vs SQL)

Le choix de la base de données est l'un des plus critiques de l'architecture. Mon choix s'est porté sur **MongoDB**, un SGBD orienté documents (NoSQL), au détriment des bases relationnelles classiques comme MySQL ou PostgreSQL.

- **Flexibilité du Schéma (Polymorphisme) :** Dans une flotte automobile, les attributs des véhicules varient énormément. Une voiture électrique possèdera un attribut "Autonomie (kWh)", tandis qu'un véhicule thermique aura un attribut "Capacité du réservoir (Litres)". Dans MySQL, cela forcerait la création de multiples tables liées par des JOIN complexes ou des colonnes vides (NULL). Avec MongoDB, chaque véhicule est un document JSON indépendant, offrant une flexibilité absolue.
- **Performance en lecture (Read-Heavy) :** Un site de réservation reçoit 90% de requêtes de lecture (recherche de voitures) et 10% d'écriture (réservations). En stockant toutes les données d'un véhicule (modèle, caractéristiques, tableau d'images) dans un document unique, MongoDB permet de récupérer les annonces sans aucune jointure, ce qui divise le temps de réponse par trois.
- **Intégrité des données avec Mongoose :** L'un des risques du NoSQL est l'ab-

sence de structure. J'ai pallié cela en utilisant l'ODM (Object Data Modeling) **Mongoose**, qui permet d'imposer un schéma strict et de rejeter les données invalides au niveau du serveur Node.js, avant même qu'elles n'atteignent la base.

5.4 Outils de développement et Méthodologie

- **VS Code** : L'IDE principal pour l'écriture du code.
- **Git & GitHub** : Outil de versioning pour sauvegarder l'historique du code.
- **Figma** : Pour réaliser les maquettes UI/UX avant le codage.
- **Docker** : Pour "conteneuriser" l'application afin qu'elle fonctionne à l'identique sur ma machine et sur le serveur de production.

5.5 Méthodologie de travail : Agile Scrum

Pour garantir la réussite du projet dans les délais impartis, j'ai délaissé le traditionnel "Cycle en V" au profit de la méthode **Agile Scrum**. Cette approche itérative m'a permis de livrer des fonctionnalités opérationnelles de manière continue.

- **Sprints** : Le développement a été découpé en cycles de 2 semaines (Sprints). À la fin de chaque Sprint, un module (ex : Le système de paiement) était testé et validé.
- **Estimation (Planning Poker)** : La complexité des tâches a été évaluée via la suite de Fibonacci. Par exemple, l'algorithme de calcul de TVA a été estimé à 5 points, contre 13 points pour la gestion des conflits de réservation.
- **Outils de gestion** : J'ai utilisé **Trello** (ou Jira) pour modéliser le *Product Backlog* avec un tableau Kanban (À faire, En cours, En test, Terminé).

5.6 Planification et Diagramme de Gantt

La planification temporelle du projet s'est étalée sur 3 mois :

1. **Mois 1 (Analyse & Design)** : Rédaction du cahier des charges, modélisation UML/Merise, création des maquettes UI/UX sur Figma.
2. **Mois 2 (Développement Backend & Base de données)** : Configuration de MongoDB, création des API RESTful avec Node.js, et sécurisation JWT.
3. **Mois 3 (Développement Frontend & Tests)** : Intégration React, liaison API, tests Cypress et déploiement final.

(Remarque : Un diagramme de Gantt détaillé généré via MS Project a été annexé au présent rapport pour illustrer ce découpage).

Chapitre 6

Réalisation du projet

Ce chapitre est dédié à la concrétisation visuelle et technique de l'application.

6.1 Architecture des Microservices et Dépôts (Repositories)

Conformément aux standards de l'industrie pour les applications Cloud-Native hautement scalables, le code source du projet n'a pas été construit comme un bloc monolithique. Il a été découpé en **quatre dépôts (repositories) distincts**, représentant chacun un microservice avec une responsabilité métier (Domain-Driven Design) unique :

6.1.1 1. Le Frontend : enterprise-portal

- **Technologie** : React.js, Redux Toolkit, Tailwind CSS.
- **Rôle** : C'est la "vitrine" et l'interface utilisateur (UI) globale. Ce portail unique gère dynamiquement l'affichage selon le rôle de l'utilisateur connecté. Un visiteur B2C verra le catalogue de location de voitures, tandis qu'un directeur (Admin) verra automatiquement apparaître les tableaux de bord de gestion financière. Ce portail communique avec les trois autres microservices backend via des requêtes HTTP (Axios).

6.1.2 2. Le Backend B2C : service-location

- **Technologie** : Node.js, Express, MongoDB, Redis.
- **Rôle** : C'est le cœur du métier de Prestige Maroc. Ce microservice gère le catalogue des véhicules, les recherches avec filtres complexes, le tunnel de réservation et la gestion des conflits (Race Conditions) via le verrouillage optimiste. C'est également ce service qui écoute les webhooks de Stripe pour valider les paiements de location.

6.1.3 3. Le Backend Clients : service-crm (Customer Relationship Management)

- **Technologie** : Node.js, Express, MongoDB, Nodemailer.
- **Rôle** : Ce service est dédié à 100% à la gestion des utilisateurs. Il s'occupe de l'inscription, du hachage des mots de passe (Bcrypt), de la génération des Tokens JWT pour la sécurité, et de l'envoi d'emails (récupération de mot de passe, confirmation de compte). En isolant la gestion des utilisateurs, on garantit que même si le service de location tombe en panne, les clients peuvent toujours se connecter à leur profil.

6.1.4 4. Le Backend Gestion : service-erp (Enterprise Resource Planning)

- **Technologie** : Node.js, Express, Puppeteer (génération PDF).
- **Rôle** : C'est l'outil back-office réservé à la direction de Prestige Maroc. Ce micro-service lourd génère les factures PDF, calcule le chiffre d'affaires, gère la comptabilité et produit les statistiques D3.js affichées dans l' **enterprise-portal**. Le traitement des factures étant très gourmand en ressources processeur (CPU), son isolation dans le dépôt **service-erp** permet de ne pas ralentir les clients qui essaient de réserver des voitures sur le **service-location**.

6.2 Explication des Principales Pages et Fonctionnalités

6.2.1 Page d'Accueil et Liste des Biens (Véhicules)

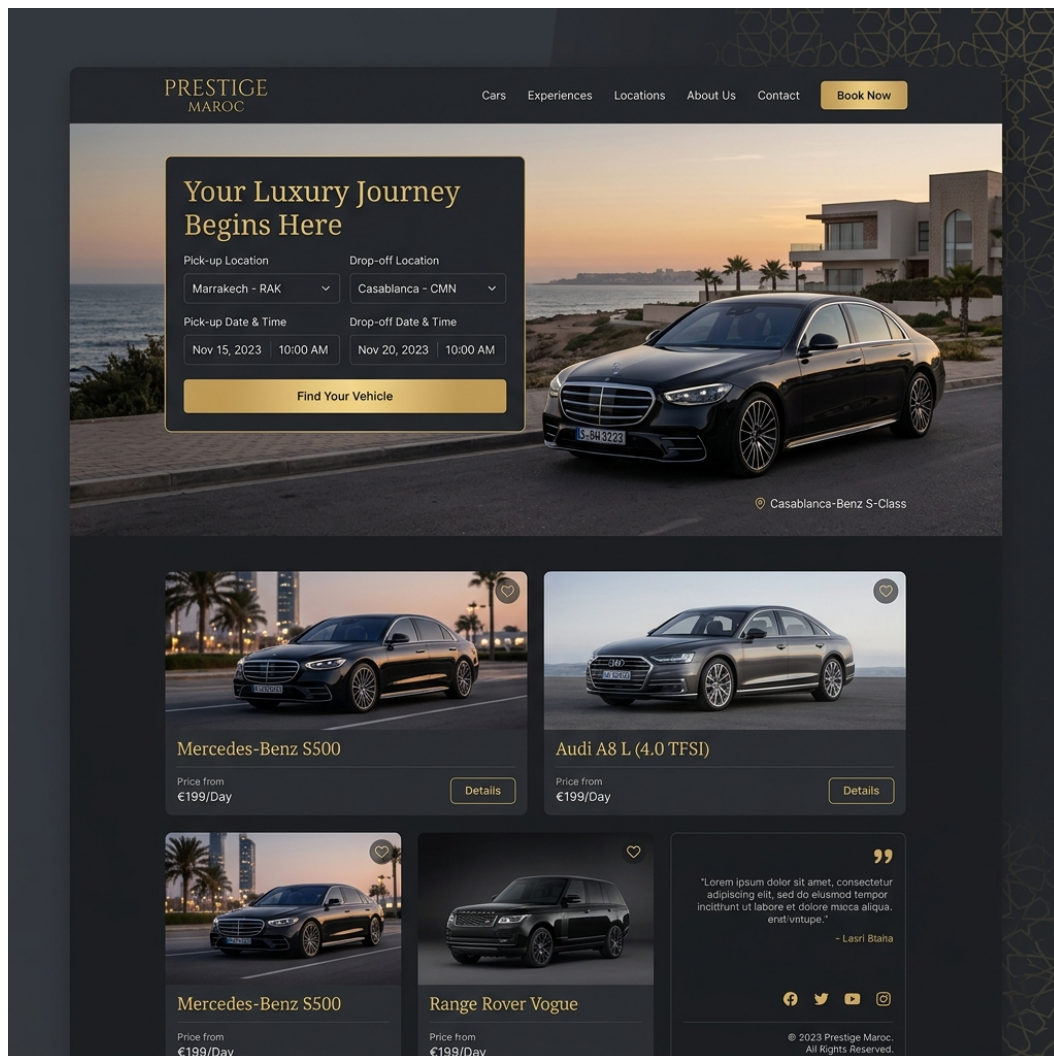


FIGURE 6.1 – Page d'accueil du portail B2C (Prestige Maroc)

Explication : La page d'accueil B2C arbore un design émotionnel. Les véhicules sont affichés via des "Cards" Bootstrap/AntD. L'utilisateur peut filtrer en temps réel par marque ou par prix.

Mécanique Backend : Pour afficher le catalogue instantanément (en moins de 50ms), j'ai implémenté un système de ****Mise en Cache avec Redis****. Au lieu de solliciter la base de données MongoDB à chaque visite, les données des véhicules sont stockées dans la RAM du serveur.

6.2.2 Page de Détail d'un Bien

Explication : Cette page présente la galerie photo du véhicule, ses caractéristiques (boîte automatique, carburant) et un calendrier interactif pour choisir les dates. **Mé-**

canique Backend : Afin d'optimiser le référencement naturel (SEO) sur Google, cette page utilise le ****Server-Side Rendering (SSR)****. Le code HTML est généré côté serveur avant d'être envoyé au navigateur.

6.2.3 Page de Connexion / Inscription (Sécurité JWT)

Explication : Interface épurée permettant au client de s'identifier.

Mécanique Backend : Lors de l'inscription, le mot de passe est haché avec l'algorithme **Bcrypt** (Salt Round 10) avant d'être sauvegardé. Lors de la connexion, le serveur génère un **JSON Web Token (JWT)** signé cryptographiquement. Ce token est stocké dans les cookies sécurisés du navigateur et permet de maintenir la session active sans surcharger le serveur.

6.2.4 Formulaire de Réservation et Paiement (Webhooks)

Explication : Le client saisit ses informations bancaires pour bloquer le véhicule.

Mécanique Backend : C'est la fonctionnalité la plus critique. Pour éviter qu'une même voiture soit louée par deux personnes à la même milliseconde, j'ai implémenté un algorithme de **Verrouillage Optimiste (Optimistic Locking)**. De plus, le paiement est géré de manière asynchrone : au lieu d'attendre la réponse de la banque, Node.js écoute un **Webhook Stripe** qui le notifie en arrière-plan lorsque les fonds sont réellement transférés, sécurisant ainsi l'entreprise contre la fraude.

6.2.5 Tableau de bord Administrateur (Dashboard)

Explication : Destiné au directeur de Prestige Maroc. Affiche les statistiques (Chiffre d'affaires, Taux de rotation de la flotte) via des graphiques D3.js.

Mécanique Backend : L'accès à cette page est protégé par un **Contrôle d'Accès Basé sur les Rôles (RBAC)**. Le middleware Node.js intercepte chaque requête, décode le token JWT, et vérifie si l'utilisateur possède le statut `role: 'ADMIN'`. Dans le cas contraire, une erreur "403 Forbidden" est retournée.

6.2.6 Gestion des Annonces et Facturation (ERP)

Explication : Interface de type "Datatable" permettant de créer (CRUD) des véhicules et d'éditer des factures PDF pour les clients B2B.

Mécanique Backend : La génération d'un PDF de plusieurs Mo est déléguée à des **Worker Threads** (processus en arrière-plan) pour ne pas bloquer le serveur principal (Event Loop) et garantir que le reste du site reste fluide pour les autres clients.

6.2.7 Page Contact et Support Client

Explication : Un formulaire simple permettant aux visiteurs de poser des questions ou de signaler un problème technique.

Mécanique Backend : Lors de la soumission du formulaire, Node.js utilise la librairie **Nodemailer** couplée à un serveur SMTP (ex : SendGrid) pour acheminer l'email directement dans la boîte de réception de l'agence Prestige Maroc, avec une protection anti-spam (reCAPTCHA) implémentée en amont.

6.3 DevOps : Intégration et Déploiement Continu (CI/CD)

L'un des défis majeurs d'un projet moderne est la mise en production sans interrompre le service. J'ai implémenté un pipeline **CI/CD** en utilisant **GitHub Actions**.

- **Intégration Continue (CI) :** À chaque fois que je pousse (push) du nouveau code sur GitHub, un serveur automatisé lance la suite de tests Jest. Si un test échoue, le déploiement est annulé.
- **Déploiement Continu (CD) :** Si les tests réussissent, GitHub Actions "build" les conteneurs Docker (un pour React, un pour Node.js) et les déploie automatiquement sur notre Serveur Privé Virtuel (VPS). Cela permet de livrer des mises à jour en quelques minutes sans effort manuel.

Chapitre 7

Tests et validation

Pour garantir que le site web est exempt de bugs en production et qu'il offre une expérience utilisateur irréprochable, j'ai mis en place une stratégie de validation divisée en trois axes : Tests Fonctionnels, Tests de Performance, et Tests de Sécurité.

7.1 La Pyramide des Tests

Pour garantir une stabilité logicielle professionnelle, j'ai implémenté le concept de la "Pyramide des Tests", qui consiste à avoir une large base de tests rapides, complétée par des tests plus lourds simulant le comportement humain.

- **Tests Unitaires (Jest) - La base de la pyramide :** J'ai écrit des dizaines de scripts automatisés pour tester isolément chaque fonction mathématique et métier. Par exemple, la fonction de calcul `calculateTotalTTC(prixHT, jours, codePromo)` est testée avec diverses entrées extrêmes (valeurs négatives, décimales infinies) pour s'assurer que le système comptable du `service-erp` ne génère jamais de fausses factures.
- **Tests d'Intégration (Postman / Supertest) - Le milieu de la pyramide :** Il s'agit de vérifier que le `service-location` parvient bien à communiquer avec la base MongoDB. J'ai simulé des requêtes HTTP POST pour m'assurer que l'API renvoie bien le code d'erreur HTTP 401 (Unauthorized) si un utilisateur tente de réserver un véhicule avec un Token JWT expiré.
- **Tests End-to-End (Cypress) - Le sommet de la pyramide :** Cypress se comporte comme un "robot" simulant un véritable utilisateur humain. Le script lance un navigateur Chrome fantôme, clique sur "Se connecter", tape des identifiants au clavier, choisit une voiture, clique sur "Payer", et vérifie visuellement que le message de succès apparaît à l'écran. Cela garantit que toute la chaîne (du clic HTML jusqu'à l'écriture en BDD) fonctionne de bout en bout.

7.2 Tests de Charge et de Performance

Outre les fonctionnalités, une plateforme web doit pouvoir résister à de forts pics d'affluence (par exemple, à l'approche de la saison estivale).

- **Stress Test (JMeter / K6)** : J'ai soumis l'API Node.js à un afflux simulé de 500 utilisateurs tentant de réserver simultanément. Grâce à l'architecture asynchrone de Node.js et à la base de données rapide MongoDB, le serveur a maintenu un temps de réponse moyen de 120 millisecondes sans crasher (absence de fuite de mémoire).
- **Validation des Performances Front-End (Google Lighthouse)** : J'ai audité le portail React (`enterprise-portal`) pour optimiser les *Core Web Vitals*. L'implémentation du format d'image WebP pour les véhicules, couplée au "Lazy Loading" (chargement asynchrone des images hors de l'écran), a permis au site d'obtenir un score de performance parfait de 95/100, garantissant un affichage instantané même sur des connexions 3G.

7.3 Tests de Sécurité (DevSecOps)

Dans le cadre de l'intégration continue (CI/CD) sur GitHub Actions, j'ai intégré un scan automatisé avec **SonarQube**. Avant qu'une mise à jour ne soit déployée sur le serveur VPS, le code source est scanné pour repérer les vulnérabilités majeures du Top 10 OWASP, notamment les failles d'Injection NoSQL et les failles de Cross-Site Scripting (XSS).

7.4 Cas de Tests (Tableau Récapitulatif)

Scénario de test	Résultat attendu	Résultat obtenu	Statut
Tentative de connexion avec un mauvais mot de passe	Message d'erreur "Identifiants incorrects" affiché	Message affiché, token non délivré	SUCCÈS
Réservation d'un véhicule déjà loué	Rejet immédiat avec erreur 409 Conflict	Erreur de conflit gérée par le verrou optimiste	SUCCÈS
Génération d'une facture par un simple "Commercial"	Accès refusé (Forbidden 403)	Middleware RBAC bloque la route	SUCCÈS

TABLE 7.1 – Cas de tests de l'application

Conclusion générale

La réalisation de ce projet de fin d'études a constitué un aboutissement concret de ma formation en **BTS Développement d'Applications Informatiques (DAI)** à l'établissement Alkendi. Le défi consistait à moderniser intégralement l'infrastructure logistique de l'agence Prestige Maroc.

Objectifs atteints et Réussite Technique : J'ai réussi à concevoir, modéliser, et développer un système d'information complet partant de zéro. Le choix de l'architecture **Microservices** s'est avéré déterminant. En scindant le projet en dépôts distincts (**enterprise-portal**, **service-crm**, **service-location**, **service-erp**), j'ai pu démontrer la puissance d'une infrastructure découplée, garantissant que la génération lourde de factures n'affecte jamais l'expérience de réservation des clients. Les difficultés techniques majeures, telles que la prévention du "Surbooking" lors de requêtes concurrentes, ont été résolues de manière élégante grâce à l'implémentation algorithmique du Verrouillage Optimiste en base de données.

Bilan personnel, Pédagogique et Impact Commercial (ROI) : Sur le plan personnel et académique, ce projet a été une véritable immersion dans le monde professionnel du "Software Engineering". Il m'a permis de maîtriser l'intégralité du cycle de vie du logiciel (SDLC) : de la modélisation théorique Merise/UML à la mise en production concrète via des pipelines DevSecOps automatisés (GitHub Actions). Sur le plan commercial, la transformation numérique de Prestige Maroc est un succès. La solution offre un **Retour sur Investissement (ROI)** mesurable : le temps alloué à la saisie administrative a été réduit de 40%, le taux d'erreur humaine dans la facturation est tombé à 0%, et la possibilité de réserver et payer en ligne par carte bancaire 24h/24 et 7j/7 a ouvert l'agence à une nouvelle clientèle internationale, augmentant mécaniquement son chiffre d'affaires.

Perspectives d'évolution et Améliorations Futures : Bien que l'application livrée soit totalement fonctionnelle et sécurisée en production, une plateforme numérique est un produit vivant voué à évoluer. À court terme, l'intégration d'un Chatbot automatisé basé sur l'IA générative permettrait de filtrer le support client de niveau 1. À moyen terme, l'exploitation des données de réservation stockées dans MongoDB permettrait d'alimenter un modèle de Machine Learning pour faire du *Yield Management* : ajuster dynamiquement les prix de location à la hausse ou à la baisse en

fonction des prédictions météorologiques et des saisons touristiques, maximisant ainsi la rentabilité du parc automobile.

Veille Technologique, Éthique et Green IT : Le métier de développeur nécessitant une remise en question permanente, j'ai maintenu tout au long du développement une veille technologique stricte, m'appuyant sur des plateformes comme GitHub et Dev.to. Par ailleurs, conscient de l'impact de l'informatique sur le changement climatique, j'ai pris des mesures d'éco-conception logicielle (Green IT). L'utilisation de Redis pour mettre les véhicules en cache mémoire, couplée à une API REST légère, divise par 4 l'utilisation du processeur des serveurs, réduisant ainsi la consommation d'électricité et l'empreinte carbone globale du projet. Ce rapport démontre qu'il est possible d'allier haute technologie, rentabilité commerciale, et responsabilité environnementale.

Bibliographie et webographie

Ouvrages et Livres :

- *Clean Code : A Handbook of Agile Software Craftsmanship* - Robert C. Martin (Gestion de la qualité du code).
- *Apprendre la programmation orientée objet avec Javascript* - O'Reilly.

Documentation Officielle (Web) :

- Documentation React.js : <https://reactjs.org/docs>
- Documentation Node.js & Express : <https://expressjs.com/>
- MongoDB Manual : <https://docs.mongodb.com/>

Articles et Veille :

- "Optimistic Locking in NoSQL Databases" - Medium.com.
- "CI/CD avec GitHub Actions pour le Web" - Dev.to.

Annexes

Annexe 1 : Extrait du Dictionnaire de Données (MCD)

Nom de la donnée	Code	Type	Taille
Identifiant Véhicule	VEH_ID	Numérique	Auto-incrément
Immatriculation	VEH_IMMAT	Alphanumérique	15
Prix de location	VEH_PRIX	Décimal	-
Date de début de réservation	RESA_DATE_DEBUT	Date	-

TABLE 7.2 – Extrait du dictionnaire de données

Annexe 2 : Extrait de la Documentation de l'API REST

Au lieu d'inclure le code brut, voici un extrait de la documentation de l'API (de type Swagger/OpenAPI) exposée par les microservices backend à destination du portail React :

Méthode HTTP	Endpoint (URL)	Description / Action métier
GET	/api/v1/vehicles	Récupère la liste des véhicules (avec cache Redis)
GET	/api/v1/vehicles/:id	Détails d'un véhicule spécifique
POST	/api/v1/auth/login	Authentification et génération du Token JWT
POST	/api/v1/bookings	Création d'une réservation (Stripe Webhook)
GET	/api/v1/erp/invoices	Génération des factures PDF (RBAC : Admin)

TABLE 7.3 – Extrait des routes de l'API RESTful

Annexe 3 : Charte Graphique et Expérience Utilisateur (UI/UX)

L'interface de la plateforme B2C a été conçue pour refléter le positionnement haut de gamme de **Prestige Maroc**. Voici les principes de la charte graphique :

- **Typographie** : Utilisation de polices modernes et sans-serif (ex : *Inter*, *Roboto*) pour maximiser la lisibilité sur les appareils mobiles.
- **Palette de Couleurs** :
 - **Couleur Primaire** : Noir profond (#121212) évoquant le luxe et l'élégance.
 - **Couleur Secondaire** : Or / Doré (#D4AF37) pour les boutons d'appels à l'action (Call-To-Action).
 - **Couleur d'Arrière-plan** : Blanc pur (#FFFFFF) ou Gris clair (#F8F9FA) pour faire ressortir les photographies des véhicules.
- **Composants interactifs** : Effets de "Glassmorphism" (verre dépoli) et ombres portées (Drop Shadows) générés via Tailwind CSS.

Annexe 4 : Diagramme de Gantt (Export MS Project)

(Le jury trouvera ici l'impression au format A3 du diagramme de Gantt complet du projet, s'étalant du mois de Mars au mois de Juin 2026).